

Executive Summary

1. Executive Summary

A penetration testing and vulnerability assessment was conducted against the Mediroza General Hospital web application to identify security weaknesses that could expose sensitive information or allow unauthorized access to protected resources.

The assessment identified several security weaknesses across authentication, access control, document protection, information disclosure, and server configuration. The most significant finding was an SQL injection vulnerability in the patient login functionality that allowed authentication to be bypassed. Following the bypass, restricted patient reports could be accessed.

Further investigation identified weaknesses in PDF password protection and sensitive metadata contained within one of the patient reports. The metadata provided an internal clue that led to an exposed `/old/` directory. Directory listing was enabled in this directory, allowing an old database backup to be discovered and downloaded. The backup contained confidential organizational information, including staff salary information and shareholder information.

The combined vulnerabilities demonstrate that weaknesses in authentication, access control, document security, and server configuration can be chained together to significantly increase the impact of an attack.

Key Recommendations

The organization should:

- Replace vulnerable SQL queries with parameterized queries or prepared statements.
- Implement strong server-side authentication and authorization controls.
- Ensure sensitive patient documents cannot be accessed without proper authorization.
- Use strong and unique protection mechanisms for encrypted documents.
- Remove unnecessary metadata from sensitive documents before distribution.
- Disable directory listing on web-accessible directories.
- Remove database backups and other sensitive files from the public web root.
- Store backups in secured locations with appropriate access controls and encryption.
- Conduct regular security assessments and configuration reviews.

The overall assessment indicates that the application requires **high-priority remediation**, particularly for the authentication bypass and publicly accessible database backup.

2. Test Parameters

2.1 Project Objective

The objective of this penetration testing and vulnerability assessment was to evaluate the security of the Mediroza General Hospital web application and identify vulnerabilities that could allow unauthorized access to protected resources or disclosure of sensitive information.

The assessment focused on identifying and demonstrating security weaknesses in authentication, authorization, input validation, document protection, information disclosure, and server configuration.

The assessment also aimed to determine the potential impact of identified vulnerabilities and provide practical recommendations for remediation.

2.2 Project Scope

The assessment was conducted against the authorized Mediroza General Hospital web application.

In-Scope Target:

`https://medirozahospital.com`

The testing was performed as a **black-box penetration test**, meaning the assessment was conducted primarily from the perspective of an external attacker without prior access to the application's source code or internal infrastructure.

Testing activities were restricted to the target domain specified in the engagement.

2.3 Project Schedule

Assessment Duration: 5 days

The testing activities covered reconnaissance, vulnerability identification, controlled exploitation, evidence collection, and reporting.

2.4 Target

Parameter	Details
Client	Mediroza General Hospital
Target	<code>https://medirozahospital.com</code>
Assessment Type	Black-box Web Application Penetration Test
Testing Environment	Authorized educational/controlled environment
Primary Application Area	Patient portal and associated resources

2.5 Limitations

The following limitations applied during the assessment:

- Testing was restricted to the authorized Mediroza Hospital domain.
- No social engineering activities were performed.
- No denial-of-service testing was performed.
- Testing was conducted within the boundaries defined by the exercise.
- Sensitive patient information was not unnecessarily collected, reproduced, or disclosed.
- Exploitation was limited to activities necessary to demonstrate the identified vulnerabilities and their potential impact.

These limitations are important because penetration tests should remain within the defined engagement scope. OWASP similarly emphasizes respecting the defined scope throughout information gathering and active testing.

2.6 Testing Methodology

The assessment followed a structured web application security testing approach consisting of:

1. **Reconnaissance and Information Gathering**
Identification of application entry points, accessible resources, metadata, and exposed directories.

2. **Authentication Testing**
Assessment of the patient login functionality and authentication controls.
3. **Input Validation and Injection Testing**
Testing of user-controlled parameters for injection vulnerabilities.
4. **Authorization and Access Control Testing**
Assessment of whether protected patient resources could be accessed without appropriate authorization.
5. **Document Security Testing**
Examination of PDF encryption and password protection.
6. **Metadata Analysis**
Inspection of document metadata for information disclosure.
7. **Configuration and Deployment Testing**
Examination of `robots.txt`, directory listing, and exposed backup files.
8. **Sensitive Information Exposure Assessment**
Analysis of the exposed database backup to determine the type of information disclosed.
9. **Evidence Collection and Reporting**
Screenshots, HTTP responses, command output, and relevant findings were documented for the final penetration-testing report.

This methodology aligns with the OWASP approach, which covers information gathering, authentication, authorization, injection, cryptography, configuration, and other web application security testing areas.

3. Findings Summary

The penetration test identified seven security vulnerabilities affecting the Mediroza General Hospital web application. The findings cover weaknesses in authentication, access control, document protection, information disclosure, and server configuration.

Ref.	Vulnerability	Location	Risk
F-01	Username enumeration on login page	/patient/login.php	Medium
F-02	SQL injection login bypass	/patient/login.php	Critical

Ref.	Vulnerability	Location	Risk
F-03	Encrypted PDFs accessible after login bypass	/patient/reports/	High
F-04	Weak PDF passwords crackable with a wordlist	patient_report_*.pdf	High
F-05	Sensitive metadata left in patient PDF files	patient_report_3.pdf	Medium
F-06	Forgotten backup folder with directory listing enabled	/old/	Critical
F-07	Confidential staff salaries and shareholder data in plain text	/old/mediroza_db_backup_2019.sql	Critical

Risk Overview

The assessment identified:

- **3 Critical** findings
- **2 High** findings
- **2 Medium** findings
- **0 Low** findings

The most significant risks were the SQL injection authentication bypass and exposure of the database backup. When combined, these weaknesses could allow an attacker to move from unauthorized application access to sensitive organizational information.

3.1 Remediation Summary

The following actions are recommended, prioritized according to the severity of the findings:

Critical priority

1. Fix the SQL injection vulnerability using parameterized queries/prepared statements.
2. Remove the database backup from the publicly accessible web directory.
3. Disable directory listing and prevent access to backup and archive directories.

4. Implement server-side authorization checks for patient reports and other protected resources.

High priority 5. Strengthen PDF protection and use unique, strong passwords where document encryption is required. 6. Ensure sensitive documents cannot be retrieved simply through predictable or directly accessible URLs.

Medium priority 7. Remove unnecessary metadata from patient documents. 8. Review login responses to prevent username/account enumeration.

This follows the general OWASP reporting approach of presenting the findings clearly, assigning risk levels, and providing actionable remediation.

3. Findings Summary

The penetration testing assessment identified seven security vulnerabilities affecting the Mediroza General Hospital web application. The findings involve authentication, access control, document security, information disclosure, and server configuration weaknesses.

Ref.	Vulnerability	Location	Risk
F-01	Username enumeration on login page	/patient/login.php	Medium
F-02	SQL injection login bypass	/patient/login.php	Critical
F-03	Encrypted PDFs accessible after login bypass	/patient/reports/	High
F-04	Weak PDF passwords crackable with a wordlist	patient_report_*.pdf	High
F-05	Sensitive metadata left in patient PDF files	patient_report_3.pdf	Medium
F-06	Forgotten backup folder with directory listing enabled	/old/	Critical

Ref.	Vulnerability	Location	Risk
F-07	Confidential staff salaries and shareholder data in plain text	/old/mediroza_db_backup_2019.sql	Critical

3.1 Risk Distribution

The assessment identified:

-  **3 Critical** vulnerabilities
-  **2 High** vulnerabilities
-  **2 Medium** vulnerabilities
- **0 Low** vulnerabilities

The most serious issues were the **SQL injection authentication bypass** and the **publicly accessible database backup**. These weaknesses could be chained with other findings to significantly increase the potential impact of an attack.

3.2 Key Findings

The assessment demonstrated that an attacker could:

1. Interact with the patient login functionality and bypass authentication through SQL injection.
2. Access restricted patient-report functionality after the authentication bypass.
3. Obtain encrypted PDF reports.
4. Recover a PDF password using the provided wordlist during the controlled assessment.
5. Extract metadata from an unlocked PDF that disclosed information about an old database backup.
6. Discover the `/old/` directory through `robots.txt`.
7. Access a directory listing containing the database backup.
8. Download the database backup and identify confidential staff and shareholder information.

3.3 Overall Assessment

The combination of these vulnerabilities represents a **significant security risk** to the application. In particular, the authentication bypass and exposed database backup demonstrate weaknesses in both application-level security controls and server-side configuration.

The findings should be prioritized according to their severity, with the **Critical findings addressed first**, followed by the High and Medium findings.

Evidence handling: Sensitive credentials, session tokens, patient medical information, national identification numbers, and other unnecessary personal information should be redacted from screenshots and the final report. OWASP specifically recommends masking sensitive information in vulnerability reports.

4. Detailed Findings

F-01 — Username Enumeration on Login Page

Severity: Medium

Affected Component: Patient Login

Location: /patient/login.php

Description

The patient login functionality was assessed to determine whether the application disclosed information that could help an attacker distinguish between valid and invalid usernames.

During testing, different username and password combinations were submitted through the login form and the resulting HTTP responses were compared.

Account enumeration occurs when an application responds differently depending on whether a supplied username exists, potentially allowing an attacker to identify valid accounts. OWASP identifies this as a security weakness because discovered usernames can facilitate further attacks against the authentication mechanism.

Evidence

The login endpoint was observed accepting POST requests containing:

```
username=<username>&password=<password>
```

Testing was performed with invalid credentials, including:

```
test / test  
invaliduser / test  
test / wrongpassword
```

The responses were captured through Burp Suite for comparison.

Evidence Screenshot:

Insert your Burp Suite screenshot showing the login request/response here.

Impact

If differences in responses allow valid accounts to be identified, an attacker could build a list of legitimate usernames and use that information to support password-guessing or other authentication attacks.

Recommendation

The application should:

- Return a consistent error message for failed authentication attempts.
- Avoid revealing whether a username exists.
- Keep response status, content and timing as consistent as practical.
- Implement rate limiting and account-protection mechanisms.
- Review password-reset and account-recovery functions for similar enumeration issues.

Risk Rating

Medium

Important accuracy note

From the evidence you've shown me earlier, your `test/test`, `invaliduser/test`, and `test/wrongpassword` requests appeared to return the same **HTTP 200 response and 2,281-byte login page**. That does **not by itself prove username enumeration**.

So in your final report, don't claim that you definitively demonstrated enumeration unless your training exercise supplied additional evidence. We can phrase F-01 as an **exercise finding requiring verification**, or remove it if the instructor expects only vulnerabilities you personally demonstrated.

That distinction will make your report more technically credible.

F-02 — SQL Injection Authentication Bypass

Severity: Critical

Affected Component: Patient Login

Location: `/patient/login.php`

Description

The patient login functionality was found to be vulnerable to SQL injection. A crafted value submitted through the username parameter caused the application to accept the request and redirect the user to the patient portal without valid credentials.

SQL injection occurs when untrusted input is incorporated into database queries without appropriate safeguards. OWASP notes that successful SQL injection can allow unauthorized access to application data and, depending on the context, substantially more severe database-level actions.

Evidence

The authentication request was intercepted using Burp Suite and modified during the authorized assessment.

The resulting response was:

```
HTTP/2 302 Found
Location: portal.php
Content-Length: 0
```

The subsequent request to the patient portal returned:

```
HTTP/2 200 OK
```

and displayed the authenticated patient portal.

Evidence Screenshot:

Insert your Burp Repeater screenshot showing the successful 302 redirect here.

Impact

Successful exploitation allowed authentication controls to be bypassed and provided access to restricted patient functionality.

This creates a significant confidentiality risk because the affected application contains sensitive healthcare-related documents.

Recommendation

The application should:

- Replace dynamically constructed SQL queries with parameterized queries/prepared statements.

- Validate and constrain user input.
- Use securely implemented authentication mechanisms.
- Ensure database accounts used by the application have only the minimum required privileges.
- Perform security testing after remediation to confirm that injection is no longer possible.

Risk Rating

Critical

Evidence placement

For **F-02**, put your strongest screenshot immediately below the Evidence section:

Screenshot: Burp Suite — SQL Injection Authentication Bypass

Make sure your screenshot does **not** expose your session cookie or other authentication secrets.

F-03 — Unauthorized Access to Patient Reports

Severity: High

Affected Component: Patient Portal / Report Downloads

Location: /patient/portal.php and download.php

Description

Following the successful authentication bypass described in F-02, the assessment obtained access to the patient portal. The portal contained links to three confidential PDF laboratory reports.

This demonstrates the impact of the authentication weakness: once authentication was bypassed, restricted patient-report functionality became accessible. Authorization controls should be enforced on protected resources and requests, rather than relying only on the login page.

Evidence

The authenticated portal displayed three report entries with download links:

```
download.php?id=1  
download.php?id=2  
download.php?id=3
```

The three PDF reports were subsequently downloaded during the authorized assessment.

Evidence to insert:

- Screenshot of the patient portal showing the three report links.
- Screenshot of the Kali Downloads folder showing the downloaded PDF files.

Do not include the actual medical contents of the reports in the report. If a screenshot shows patient medical information, crop or redact it.

Impact

An attacker able to bypass authentication could potentially access confidential patient documents. Because healthcare reports may contain sensitive personal and medical information, unauthorized disclosure could result in serious privacy, regulatory, and reputational consequences.

Recommendation

The application should:

- Enforce authorization checks on every patient-report request.
- Verify that the authenticated user is authorized to access the specific requested report.
- Avoid relying solely on predictable numeric identifiers such as `id=1`, `id=2`, and `id=3`.
- Apply server-side access controls to the download endpoint.
- Return an appropriate denial response when authorization fails.
- Log and monitor unauthorized access attempts.

OWASP recommends that authorization be checked for each protected resource and that applications follow a deny-by-default approach.

Risk Rating

High

F-04 — Weak PDF Password Protection

Severity: High

Affected Files: patient_report_1.pdf, patient_report_2.pdf, patient_report_3.pdf

Description

The downloaded patient reports were protected with PDF encryption. During the authorized exercise, the built-in wordlist failed to recover the password for Report 3. The instructor-provided JTR wordlist subsequently recovered the password, demonstrating that the document password was susceptible to wordlist-based guessing.

The recovered password was then used with `qpdf` to create an unlocked copy of Report 3 for the required metadata analysis.

Evidence

The following command successfully decrypted Report 3 during the exercise:

```
qpdf --password='[REDACTED]' --decrypt patient_report_3.pdf report3_open.pdf
```

The resulting file was verified with:

```
ls -l report3_open.pdf
```

and later analyzed using:

```
pdftinfo report3_open.pdf
```

The original report was also confirmed to be encrypted using:

```
qpdf --is-encrypted patient_report_3.pdf
```

which returned exit status 0.

Evidence to insert:

- Screenshot showing the JTR wordlist result.
- Screenshot showing successful `qpdf` decryption.
- Screenshot showing `qpdf --is-encrypted` or the PDF encryption information.

Do not put the recovered password in the final report.

Impact

Weak document passwords can reduce the effectiveness of encryption and allow an attacker who obtains a protected PDF to recover its contents through password-guessing techniques.

Recommendation

- Use strong, randomly generated passwords for document encryption.
- Avoid predictable or dictionary-based passwords.
- Where possible, use centralized access control rather than relying on PDF passwords to protect confidential information.
- Review the organization's document-generation process to ensure strong encryption settings are consistently applied.

Risk Rating

High

F-05 — Sensitive Information Disclosure Through PDF Metadata

Severity: Medium

Affected Component: Patient PDF Report

Affected File: patient_report_3.pdf

Description

During the assessment, metadata from the unlocked PDF report was examined using `exiftool`. The document contained information that was not necessary for the intended recipient and that revealed internal details about the organization's environment.

OWASP notes that metadata in generated documents such as PDFs can disclose information about the software, technology, internal paths, or other implementation details and can assist further reconnaissance.

Evidence

The following command was used:

```
exiftool report3_open.pdf
```

The metadata included:

Author: j.malik
Comments/Creator: DB backup moved to /old before site migration, do not delete

The metadata therefore disclosed:

- An internal user identifier: j.malik
- The existence of an old database backup
- The location of the backup: /old/

The /old/ clue was subsequently confirmed through robots.txt, which listed:

Disallow: /old/

The /old/ directory was then found to expose the database backup.

Evidence Screenshot:

Insert your exiftool report3_open.pdf screenshot here.

Impact

The metadata disclosure provided useful reconnaissance information that helped identify a potentially sensitive backup location.

An attacker could use such information to understand aspects of the application's internal environment and identify additional targets for investigation.

Recommendation

- Remove unnecessary metadata from sensitive documents before they are made available to users.
- Configure the document-generation system to minimize metadata.
- Review existing documents for accidentally exposed internal information.
- Avoid placing internal operational notes, usernames, file paths, or system information in user-accessible documents.
- Include metadata review as part of the organization's secure document-release process.

Risk Rating

Medium

F-06 — Exposed Backup Directory With Directory Listing Enabled

Severity: Critical

Affected Component: Web Server

Location: /old/

Description

A publicly accessible /old/ directory was identified during reconnaissance. Directory listing was enabled, allowing files within the directory to be viewed and downloaded directly.

The directory was initially identified through the `robots.txt` file and subsequently accessed during the authorized assessment.

OWASP identifies old, backup, and unreferenced files as a significant source of information leakage because they can contain sensitive application or database information.

Evidence

The following command was used to inspect the site's `robots.txt`:

```
curl https://medirozahospital.com/robots.txt
```

The response included:

```
Disallow: /old/
```

The directory was then accessed with:

```
curl https://medirozahospital.com/old/
```

The server returned a directory listing containing:

```
mediroza_db_backup_2019.sql
```

The database backup was successfully downloaded during the authorized assessment.

Evidence Screenshots:

1. `robots.txt` showing /old/
2. /old/ directory listing
3. Kali Downloads folder showing the downloaded SQL backup

Impact

An attacker could discover and download files that should not be publicly accessible. In this case, the exposed backup contained sensitive organizational information.

The issue therefore creates a significant confidentiality risk and can provide attackers with information that may facilitate further compromise.

Recommendation

- Remove backup files from publicly accessible web directories.
- Store backups outside the web server's document root.
- Disable directory listing.
- Implement access controls for backup repositories.
- Encrypt sensitive backups at rest.
- Establish a process for securely deleting obsolete backups.
- Regularly scan production web directories for backup and temporary files.

Risk Rating

Critical

F-07 — Sensitive Information Exposure in Database Backup

Severity: Critical

Affected Component: Publicly accessible database backup

Location: /old/mediroza_db_backup_2019.sql

Description

A database backup file was discovered within a publicly accessible web directory. The backup was available for direct download without authentication or additional access controls.

The file contained SQL statements capable of recreating database tables and included sensitive organizational information.

Evidence

The exposed backup was discovered in the /old/ directory:

```
mediroza_db_backup_2019.sql
```

The file was downloaded during the authorized assessment and examined locally.

The `staff` table contained information including:

- Staff names
- Job titles
- Departments
- Monthly salaries

The `shareholders` table contained information including:

- Shareholder names
- Share ownership percentages
- Share classes

The database contents also allowed the metadata clue `j.malik` to be correlated with **Jameel Malik**, identified in the staff records as an **IT Systems Administrator**.

Evidence Screenshots:

1. `/old/` directory showing the exposed database backup.
2. Downloaded SQL backup file.
3. Relevant `staff` table entries.
4. Relevant `shareholders` table entries.

Sensitive information such as national identification numbers, telephone numbers, credentials, hashes, or other unnecessary personal information should be redacted from the final report.

Impact

An unauthenticated attacker could download the exposed database backup and obtain confidential organizational and personnel information.

The exposure could also provide an attacker with valuable information about the organization's internal structure and increase the effectiveness of subsequent attacks.

Because the backup was publicly accessible, the vulnerability represents a significant confidentiality risk.

Recommendation

- Remove the database backup from the publicly accessible web directory immediately.
- Store database backups outside the web server's document root.

- Restrict backup access using strong authentication and authorization.
- Encrypt sensitive backups at rest.
- Delete obsolete backups securely.
- Disable directory listing on web servers.
- Establish a backup-management policy covering storage, access, retention, and secure deletion.
- Regularly scan production environments for accidentally exposed backup files such as .sql, .bak, .zip, and configuration files.

Risk Rating

Critical

5. Recommendations and Remediation

The following actions are recommended to reduce the identified security risks:

1. **Fix SQL Injection:** Use prepared statements and parameterized queries for all database operations.
2. **Strengthen Authentication:** Implement secure authentication controls, consistent login responses, rate limiting, and protection against account enumeration.
3. **Enforce Authorization:** Verify that users are authorized to access each patient report before allowing downloads.
4. **Strengthen PDF Protection:** Use strong, unique passwords and appropriate encryption for sensitive documents.
5. **Remove Sensitive Metadata:** Strip unnecessary usernames, internal notes, file paths, and system information from documents.
6. **Secure Backup Files:** Remove database backups from publicly accessible directories and store them in protected locations.
7. **Disable Directory Listing:** Disable directory browsing and regularly check web directories for exposed backup or temporary files.
8. **Protect Sensitive Data:** Encrypt confidential database backups and restrict access based on least privilege.

Remediation should be prioritized according to the severity of the findings, with the **Critical vulnerabilities addressed first**, followed by High and Medium findings.

Conclusion

The penetration testing assessment identified several security weaknesses in the Mediroza General Hospital web application, including authentication, access control, document security, information disclosure, and exposed backup files.

The most significant risks were the SQL injection authentication bypass and the publicly accessible database backup. These weaknesses could allow unauthorized access to sensitive information.

Prompt remediation of the Critical and High-risk findings is recommended. Following remediation, a security re-test should be performed to verify that the identified vulnerabilities have been effectively resolved.